Nutan Maharashtra Vidya Prasarak Mandal's

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY

ESTD : 1906

Department of Information Technology

LABORATORY MANUAL

SUBJECT: Artificial Intelligence & Machine Learning Lab

[SUBJECT CODE: PCC-305-ITT]

CLASS: T.E

PREPARED BY: **APPROVED BY:**

Vision and Mission of the Institute

1. **Vision of the Institute-** To be a notable institution for providing quality technical education and ensuring ethical, moral and holistic development of students.
2. **Mission of the Institute**- To nurture engineering graduates with state of the art competence, professionalism and problem solving skills to serve needs of industry as well as society.

Vision and Mission of the Department

Department Vision:

To be a distinguished and competent education provider in the field of Information Technology by nurturing technically proficient, ethically responsible and socially aware professionals

Department Mission:

- Imparting technical knowledge and practical skills in Information Technology to prepare students for successful careers in industry, academia and entrepreneurship.
- Fostering ethical values, professional responsibility, and effective Communication in students to help them become responsible IT professionals.
- Encouraging innovation, research, and problem-solving abilities through industry-relevant projects, internships and collaborative learning.
- Cultivating awareness of societal needs and sustainability, promoting the development of socially responsible and globally competent graduates.

Program Outcomes

PO1 – Engineering Knowledge

Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2 – Problem Analysis

Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3 – Design / Development of Solutions

Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4 – Conduct Investigations of Complex Problems

Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5 – Engineering Tool Usage

Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6 – The Engineer and The World

Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7 – Ethics

Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8 – Individual and Collaborative Team Work

Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9 – Communication

Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10 – Project Management and Finance

Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11 – Life-Long Learning

Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Program Educational Objectives (PEOs)

PEO1: To build a foundation in Information Technology with the technical knowledge and practical skills required to succeed in industry, higher education or entrepreneurial ventures.

PEO2: To demonstrate ethical behavior, professional responsibility and effective communication in their careers, contributing positively to their workplace and society.

PEO3: To engage in innovative practices and apply critical thinking and problem-solving abilities to develop IT solutions for real-world challenges through continuous learning and collaboration.

PEO4: To exhibit awareness of societal, environmental and global issues and will contribute to sustainable development as socially responsible and globally competent professionals.

Program Specific Outcomes (PSOs)

PSO 1 - Technical Proficiency and Problem Solving
An ability to apply theoretical concepts and practical knowledge of Information Technology to analyze, design, develop, and manage information processing systems and applications. Graduates should be capable of identifying and defining appropriate computing infrastructure and operational requirements for solving real-world problems, and working effectively on large-scale computing systems.

PSO 2 - Professionalism, Ethics, and Communication
An understanding of professional, business, and ethical responsibilities, including legal, security, and social issues related to IT. Graduates will practice effective communication and decision-making skills, using appropriate technology to handle professional responsibilities and contribute to business processes in a socially responsible manner.

Course Outcomes (CO)

Course Outcome	Statement
	At the end of the course, a student will be able to
CO1	Implement intelligent agents, search algorithms and knowledge representation techniques for AI-based problem solving.
CO2	Apply data preprocessing, exploratory data analysis and supervised learning algorithms for prediction and classification tasks.
CO3	Develop and evaluate machine learning models using appropriate performance metrics and validation techniques.
CO4	Implement clustering, association analysis and dimensionality reduction techniques for pattern discovery in datasets.
CO5	Build optimized AI/ML solutions using advanced learning algorithms and apply them to real-world applications.

List of Experiments with Mapping

Sr. No.	Title of Assignment	CO	PO
	PART A		
1	**PR1: Intelligent Agents and PEAS Representation** Design and implement a simple intelligent agent for a real-world application such as a chatbot, smart home controller or recommendation assistant. Demonstrate the PEAS (Performance Measure, Environment, Actuators, Sensors) representation and agent behavior.	CO1	PO1, PO2, PO3, PO5
2	**PR2: Uninformed Search Algorithms** Implement Breadth First Search (BFS) and Depth First Search (DFS) algorithms for solving route-finding or maze navigation problems. Compare their performance based on path length, nodes explored and execution time.	CO1, CO2	PO1, PO2, PO3, PO5
3	**PR3: Informed Search Algorithms** Implement Greedy Best First Search and A* Search algorithms for shortest path finding. Analyze optimality, completeness, and efficiency of both approaches.	CO1, CO2	PO1, PO2, PO3, PO5
4	**PR4: Knowledge Representation and Expert System** Develop a rule-based expert system using propositional logic and implement Forward Chaining and Backward Chaining mechanisms for intelligent decision making.	CO1, CO2	PO1, PO2, PO3, PO5
	PART B		
5	**PR5: Data Preprocessing and Exploratory Data Analysis** Download a standard dataset (Heart Disease, Diabetes or Titanic dataset) and perform: a. Data cleaning b. Missing value handling c. Feature scaling d. Encoding techniques e. Statistical analysis f. Data visualization using histograms and boxplots Split the dataset into training and testing sets.	CO2	PO1, PO2, PO3, PO5
6	**PR6: Regression Techniques** Implement Linear Regression and Polynomial Regression on a suitable dataset such as California Housing or House Price Prediction dataset. Evaluate model performance using: a. Mean Squared Error (MSE) b. Mean Absolute Error (MAE) c. R^2 Score Compare the results obtained by both techniques.	CO2, CO3	PO1, PO2, PO3, PO5
7	**PR7: Classification using Logistic Regression and K-Nearest Neighbors** Perform data preprocessing and implement: a. Logistic Regression b. K-Nearest Neighbors (KNN) Evaluate the models using: a. Accuracy b. Precision c. Recall d. F1-Score e. Confusion Matrix Analyze and compare the results.	CO2, CO3	PO1, PO2, PO3, PO5
8	**PR8: Classification using Decision Tree and Naïve Bayes** Implement Decision Tree and Naïve Bayes classifiers on a suitable dataset. Compare classifier performance using standard evaluation metrics and identify the most suitable model for the selected dataset.	CO2, CO3	PO1, PO2, PO3, PO5
	PART C		

9	**PR9: Clustering Techniques** **Implement:** a. K-Means Clustering b. Hierarchical Clustering Use the Elbow Method and Dendrogram for determining the optimal number of clusters. Analyze the generated clusters using a real-world dataset.	CO4	PO1, PO2, PO3, PO5
10	**PR10: Dimensionality Reduction using Principal Component Analysis (PCA)** Apply PCA on a high-dimensional dataset. Compare: a. Number of features before and after PCA b. Visualization of reduced dimensions c. Impact on classification performance	CO4	PO1, PO2, PO3, PO5
11	**PR11: Advanced Machine Learning and Model Optimization** Implement and compare: a. Support Vector Machine (SVM) b. Random Forest c. AdaBoost Perform: a. Cross Validation b. Hyperparameter Tuning using Grid Search Evaluate and compare model performance before and after optimization.	CO5	PO1, PO2, PO3, PO5
12	**PR12: AI/ML-Based Real-World Application (Mini Project)** Disease Prediction System Fraud Detection System Customer Segmentation Recommendation System Student Performance Prediction Crop Yield Prediction The project should include: Problem Definition Dataset Collection Data Preprocessing Model Development Performance Evaluation Model Optimization Result Analysis and Documentation	CO1, CO5	PO1, PO2, PO3, PO4, PO5, PO11

Rubrics for Evaluation

Sr. No	Evaluation Criteria	Marks for each Criteria	Rubrics
1	Timely submission	5 or 10	Punctuality reflects the work ethic. Students should reflect that work ethic by completing the lab assignments and reports in a timely manner.
2	Journal Presentation	5 or 10	Students are expected to prepare the journal. The journal presentation of the course should be complete, clear, and understandable.
3	Performance	5 or 10	After the performance, the students should have good knowledge of the experiment.
4	Understanding	5 or 10	The student should be able to explain the methodology used for designing and developing the program/solution. Students should clearly understand the purpose of the assignment and its outcome.
5	Oral	5 or 10	The student should be able to answer the questions related to the lab assignments.

Group(A)

PRACTICAL NO. 1

INTELLIGENT AGENTS AND PEAS REPRESENTATION

Title: Intelligent Agents and PEAS Representation

1. AIM

To design and implement a simple **intelligent agent** for a real-world application and represent its working using the **PEAS framework** consisting of **Performance Measure, Environment, Actuators, and Sensors**.

2. OBJECTIVES

After completing this practical, students will be able to:

1. Understand the concept of an intelligent agent.
2. Identify different components of an intelligent agent.
3. Represent an intelligent agent using the PEAS framework.
4. Understand the interaction between an agent and its environment.
5. Implement a simple rule-based intelligent agent using Python.
6. Observe the behavior of an agent for different inputs.

3. THEORY

3.1 Intelligent Agent

An **intelligent agent** is a system that perceives its environment through **sensors** and takes suitable actions through **actuators** to achieve a specific goal.

The basic working of an intelligent agent can be represented as:

Environment → Sensors → Agent → Actuators → Environment

For example, consider a **smart home temperature controller**.

- The temperature sensor observes the room temperature.
- The agent processes the temperature.
- If the temperature is high, it switches ON the air conditioner.
- If the temperature is comfortable, it switches OFF the air conditioner.

  Thus, the agent takes an appropriate action based on the information received from the environment.

4. PEAS REPRESENTATION

PEAS stands for:

Component	Meaning	Description
P	Performance Measure	Criteria used to measure the success of the agent
E	Environment	Surroundings in which the agent operates
A	Actuators	Mechanisms through which the agent performs actions
S	Sensors	Mechanisms through which the agent receives information

5. EXAMPLE: SMART HOME TEMPERATURE CONTROLLER

Consider an intelligent agent that automatically controls room temperature.

PEAS Description

PEAS Component	Description
Performance Measure	Maintain comfortable temperature, minimize energy consumption
Environment	Room, temperature, people, air conditioner
Actuators	AC ON, AC OFF, increase/decrease cooling

Sensors	Temperature sensor, humidity sensor, motion sensor

Agent Behavior

The agent follows rules such as:

- If temperature > 28°C → Turn ON AC.
- If temperature < 24°C → Turn OFF AC.
- If temperature is between 24°C and 28°C → Maintain current state.

6. ALGORITHM

Algorithm: Smart Home Temperature Controller

Step 1: Start.

Step 2: Read the room temperature from the sensor.

Step 3: Compare the temperature with the predefined threshold values.

Step 4: If temperature is greater than 28°C, turn ON the AC.

Step 5: If temperature is less than 24°C, turn OFF the AC.

Step 6: Otherwise, maintain the current state.

Step 7: Display the sensor input and agent action. **Step 8:** Stop.

7. SAMPLE TEST CASES

Temperature	Agent Perception	Agent Action	AC Status
32°C	High	Turn ON AC	ON
29°C	High	Turn ON AC	ON
26°C	Normal	Maintain current state	OFF/ON
23°C	Low	Turn OFF AC	OFF

20°C	Low	Turn OFF AC	OFF

8. OBSERVATION

The intelligent agent successfully perceives the room temperature and performs an appropriate action according to predefined rules.

It can:

- Detect high temperature.
- Turn ON the AC when required.
- Detect low temperature.
- Turn OFF the AC when required.
- Maintain the current state when the temperature is within the comfortable range.

9. APPLICATIONS OF INTELLIGENT AGENTS

Intelligent agents are used in many real-world applications, such as:

1. **Chatbots** – interact with users and provide responses.
2. **Smart Homes** – control lights, fans, AC, and security systems.
3. **Recommendation Systems** – recommend movies, products, or songs.
4. **Self-driving Cars** – perceive road conditions and make driving decisions.
5. **Virtual Assistants** – perform tasks based on user commands.
6. **Robotics** – navigate and perform tasks in physical environments.
7. **Healthcare Systems** – assist in diagnosis and monitoring.

10. CONCLUSION

In this practical, we studied the concept of an **intelligent agent** and its interaction with the environment. We represented a real-world application using the **PEAS framework** and implemented a simple rule-based intelligent agent in Python. This practical helps in understanding how intelligent agents perceive information, make decisions, and perform suitable actions.

PRACTICAL NO.: 2

UNINFORMED SEARCH ALGORITHMS

Title: Implementation of Breadth First Search (BFS) and Depth First Search (DFS) .

1. AIM

To study and implement **Breadth First Search (BFS)** and **Depth First Search (DFS)** algorithms for route finding or maze navigation and compare their performance based on **path length, number of nodes explored, and execution time**.

2. OBJECTIVES

1. To understand the concept of uninformed search algorithms.
2. To study BFS and DFS algorithms.
3. To understand the working of queue and stack in search algorithms.
4. To compare BFS and DFS based on their performance.
5. To identify the applications, advantages, and limitations of BFS and DFS.

3. THEORY

3.1 UNINFORMED SEARCH

Uninformed search, also called **blind search**, is a search technique in which the algorithm does not have additional information about the distance or direction of the goal.

The search is performed using only the information available in the problem definition. Important uninformed search algorithms include:

- Breadth First Search (BFS)
- Depth First Search (DFS)
- Uniform Cost Search
- Depth Limited Search
- Iterative Deepening Search

BFS and DFS are fundamental graph and tree traversal techniques.

4. BREADTH FIRST SEARCH (BFS)

Breadth First Search explores nodes level by level. It first visits the starting node, then all its immediate neighbors, followed by nodes at the next level. BFS uses a **Queue (FIFO – First In First Out)** data structure. **Example**

Consider the following graph:



Starting from **A**, BFS explores the nodes approximately as:

A → B → C → D → E → F

If the goal is **F**, BFS can find the route:

A → C → F

This is the shortest path in terms of the number of edges for an unweighted graph.

BFS ALGORITHM

Step 1: Start with the initial node.

Step 2: Insert the initial node into a queue.

Step 3: Mark the node as visited.

Step 4: Remove the first node from the queue.

Step 5: Check whether the current node is the goal.

Step 6: If it is not the goal, add all unvisited neighboring nodes to the queue.

Step 7: Mark the newly added nodes as visited.

Step 8: Repeat Steps 4–7 until the goal is found or the queue becomes empty.

Step 9: Display the path and search information.

Step 10: Stop.

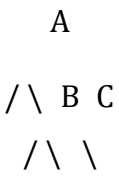
5. DEPTH FIRST SEARCH (DFS)

Depth First Search explores a path as deeply as possible before backtracking and exploring another path.

DFS uses a **Stack (LIFO – Last In First Out)** or recursion.

Example

For the same graph:



DFS may explore the nodes as:

A → B → D → E → C → F

The exact order depends on the order in which neighboring nodes are considered. DFS does not guarantee the shortest path.

DFS ALGORITHM

Step 1: Start with the initial node.

Step 2: Insert the initial node into a stack.

Step 3: Mark the node as visited.

Step 4: Remove the top node from the stack.

Step 5: Check whether the current node is the goal.

Step 6: If it is not the goal, add all unvisited neighboring nodes to the stack.

Step 7: Mark newly added nodes as visited.

Step 8: Continue the process until the goal is found or the stack becomes empty.

Step 9: Display the path and search information.

Step 10: Stop.

6. COMPARISON OF BFS AND DFS

arameter	BFS	DFS
ull Form	readth First Search	epth First Search
ata Structure	ueue	tack
Vorking	evel by level	epth-wise
hortest Path	uaranteed for unweighted graph	ot guaranteed
Iemory Usage	enerally higher	enerally lower
ompleteness	omplete for finite branching	Iay not be complete in infinite/deep earch spaces
ime Complexity	(V + E)	(V + E)
pace Complexity	(V)	(V)
uitable For	hortest route/minimum teps	Iaze/backtracking problems

Here, **V** represents the number of vertices/nodes and **E** represents the number of edges.

7. PERFORMANCE COMPARISON

BFS and DFS can be compared using the following parameters:

7.1 PATH LENGTH

For an unweighted graph, BFS finds the path with the minimum number of edges. DFS may find a path that is longer than the shortest available path.

7.2 NODES EXPLORED

The number of nodes explored depends on the structure of the graph and the location of the goal. BFS may explore several nodes at the same level before reaching the goal, whereas DFS may reach a deep goal quickly if it chooses the correct branch.

7.3 EXECUTION TIME

Execution time depends on:

- Number of nodes
- Number of edges
- Structure of the graph
- Position of the goal
- Order of node exploration
- Computer hardware and software environment

For theoretical analysis, both BFS and DFS have a time complexity of **O(V + E)** for graph traversal.

8. APPLICATIONS

Applications of BFS

1. Finding the shortest route in an unweighted network.
2. Finding minimum steps in puzzles and games.
3. Social network analysis.
4. Network broadcasting.
5. Web crawling.
6. Finding nearby locations in a map.

APPLICATIONS OF DFS

1. Maze navigation.
2. Backtracking problems.
3. File and directory traversal.
4. Cycle detection in graphs.
5. Topological sorting.
6. Finding connected components.

9. ADVANTAGES

ADVANTAGES OF BFS

1. Finds the shortest path in an unweighted graph.
2. Simple and systematic search technique.
3. Complete for finite search spaces.
4. Useful for minimum-step problems.
5. Suitable For Level-Wise Exploration.

ADVANTAGES OF DFS

1. Requires comparatively less memory in many search situations.
2. Easy to implement using a stack or recursion.
3. Effective for maze and backtracking problems.
4. Can reach deep solutions without exploring all shallow nodes.
5. Useful for graph connectivity and traversal problems.

10. LIMITATIONS

Limitations of BFS

1. Requires more memory for large search spaces.
2. May explore many unnecessary nodes.
3. Can become inefficient when the solution is very deep.
4. Not suitable for very large graphs when memory is limited.

Limitations of DFS

1. Does not guarantee the shortest path.
2. May follow an incorrect path for a long time.
3. Recursive DFS may cause stack overflow for very deep graphs.
4. Performance depends on the order of node exploration.
5. Requires a visited mechanism to avoid repeated exploration in graphs containing cycles.

11. OBSERVATIONS

BFS and DFS search algorithms were studied for route finding and maze navigation. Their performance was compared based on **path length, nodes explored, and execution time**.

12. CONCLUSION

In this practical, we studied the two important uninformed search algorithms, BFS and DFS. BFS explores nodes level by level using a queue and is suitable when the shortest path is required in an unweighted graph. DFS explores nodes deeply using a stack or recursion and is useful for maze solving and backtracking problems.

Thus, BFS is preferred for shortest-path problems, whereas DFS is useful for deep exploration and problems involving backtracking. The selection of the algorithm depends on the structure of the search space, memory availability, and the required type of solution.

PRACTICAL NO.: 3

INFORMED SEARCH ALGORITHM

Title: Implementation of **A* Search Algorithm** for Shortest Path Finding

1. AIM

To study and implement the **A* (A-star) Search algorithm** for shortest path finding and analyze its **optimality, completeness, and efficiency**.

2. OBJECTIVES

1. To understand the concept of informed search.
2. To study the working of the A* Search algorithm.

3. To understand the role of heuristic functions in informed search.
4. To find an optimal path between a start node and a goal node.
5. To analyze the optimality, completeness, and efficiency of A* Search.

3. THEORY

3.1 Informed Search

Informed search, also known as **heuristic search**, uses additional information about the problem to guide the search toward the goal.
Unlike uninformed search algorithms such as BFS and DFS, informed search algorithms use a **heuristic function** to estimate how close a node is to the goal.
A commonly used informed search algorithm is **A* Search**.

4. A* SEARCH ALGORITHM

A* Search is a popular informed search algorithm used to find the shortest or least-cost path between a starting node and a goal node.
A* combines:

- The actual cost required to reach a node.
    - The estimated cost from that

node to the goal. It uses the evaluation function:

f(n) = g(n) + h(n)

Where:

- **f(n)** = Total estimated cost of the solution through node *n*
- **g(n)** = Actual cost from the start node to node *n*
- **h(n)** = Estimated cost from node *n* to the goal

The node having the **lowest f(n)** value is selected for expansion.

5. HEURISTIC FUNCTION

A **heuristic function h(n)** estimates the cost of reaching the goal from the current node. For example, in a map-based route-finding problem, the straight-line distance between the current location and the destination can be used as a heuristic.

A good heuristic helps A* reach the goal efficiently by reducing unnecessary exploration.

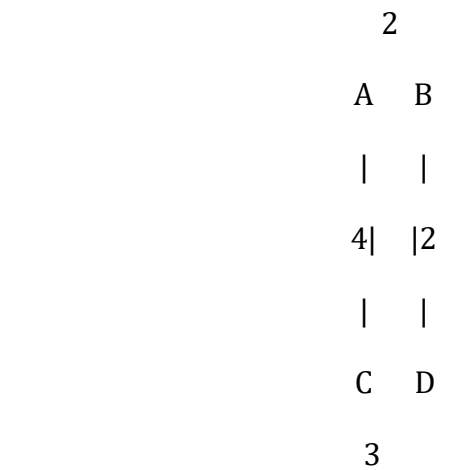
Properties of a Good Heuristic

1. It should provide a reasonable estimate of the remaining cost.
2. It should not overestimate the actual cost when optimality is required.
3. It should help reduce the number of nodes explored.
4. It should be computationally efficient.

A heuristic that never overestimates the actual cost is called an **admissible heuristic**.

6. EXAMPLE

Consider the following simplified graph:



Suppose:

- Start node = **A**
- Goal node = **D**

For every node, A* calculates:

f(n) = g(n) + h(n)

Example heuristic values:

			f(n) = g(n)+h(n)
			4
			4
			7
			4

A* selects the node with the smallest **f(n)** value and continues the search until the goal is reached.

The shortest route is:

A → B → D

Total path cost:

2 + 2 = 4

Thus, A* can efficiently find an optimal path when an appropriate admissible heuristic is used.

7. A* SEARCH ALGORITHM

Step 1: Start with the initial node.

Step 2: Calculate its heuristic value and set:

f(n) = g(n)

+ h(n) Step 3: Add the starting node to the OPEN list.

Step 4: Select the node having the lowest f(n) value from the OPEN list.

Step 5: Move the selected node to the CLOSED list.

Step 6: If the selected node is the goal node, reconstruct and return the path.

Step 7: Otherwise, generate its neighboring nodes.

Step 8: Calculate the cost values **g(n), h(n), and f(n)** for each neighboring node.

Step 9: Add the appropriate unvisited or lower-cost nodes to the OPEN list.

Step 10: Repeat Steps 4–9 until the goal is reached or no nodes remain.

Step 11: Display the shortest path and total path cost.

Step 12: Stop.

8. OPTIMALITY OF A* SEARCH

A* Search is **optimal** when the heuristic function is **admissible**, meaning that it never overestimates the actual cost of reaching the goal.

For example, if the actual cost from a node to the goal is 10, the heuristic should estimate a value of **10 or less**. An admissible heuristic ensures that A* does not ignore a potentially better path.

Therefore:

A* + Admissible Heuristic → Optimal Solution

For graph search, additional consistency conditions are commonly used to guarantee optimality efficiently.

9. COMPLETENESS OF A* SEARCH

An algorithm is **complete** if it is guaranteed to find a solution whenever a solution exists, subject to the assumptions of the search problem.

A* is complete under standard conditions such as:

- Positive step costs.
- A finite branching factor.
- The existence of a solution with finite cost.

Thus, under suitable conditions, A* will eventually find a solution if one exists.

10. EFFICIENCY OF A* SEARCH

A* is generally more efficient than uninformed search when a good heuristic is available. The heuristic provides additional information that guides the search toward the goal.

Efficiency depends on:

1. Quality of the heuristic.
2. Number of nodes in the search space.
3. Branching factor.
4. Cost of individual actions.
5. Distance between the start and goal.

A poor heuristic may cause A* to explore many nodes, while a good heuristic can significantly reduce the search effort.

11. PERFORMANCE ANALYSIS

A* can be analyzed using the following parameters:

11.1 Path Cost

A* aims to find a path with the minimum total cost when the heuristic satisfies the required conditions.

11.2 Nodes Explored

A good heuristic can reduce the number of nodes explored compared with uninformed search methods.

11.3 Execution Time

Execution time depends on the size of the search space, heuristic quality, and number of nodes explored.

11.4 Memory Usage

A* maintains information about generated nodes, their costs, and paths. Therefore, it can require significant memory for large search spaces.

12. APPLICATIONS

A* Search is widely used in:

1. Route finding in maps and navigation systems.
2. Maze solving.
3. Robot path planning.
4. Video game character navigation.
5. Network routing.
6. Puzzle solving.
7. Autonomous vehicle navigation.
8. Artificial intelligence planning systems.

13. ADVANTAGES

1. Finds an optimal path when an appropriate admissible heuristic is used.
2. More goal-directed than uninformed search algorithms.
3. Can be significantly faster with a good heuristic.
4. Combines actual path cost and estimated remaining cost.
5. Widely applicable to route finding and path-planning problems.
6. Flexible because different heuristic functions can be used for different problems.

14. LIMITATIONS

1. Can require large amounts of memory.
2. Performance depends heavily on the quality of the heuristic.
3. A poor heuristic may result in exploration of many nodes.
4. Designing an effective heuristic can be difficult.
5. Computational cost increases for very large search spaces.

6. An overestimating heuristic may cause the algorithm to lose its optimality guarantee.

15. OBSERVATION

The **A* Search algorithm** was studied for shortest path finding. The algorithm uses the evaluation function **f(n) = g(n) + h(n)** to select the most promising node and efficiently search for the goal.

16. CONCLUSION

In this practical, we studied the **A* Search algorithm** for shortest path finding using **f(n) = g(n)**

+ h(n). A* provides an optimal solution when an **admissible heuristic** is used and efficiently guides the search toward the goal. It is widely used in **route finding, maze navigation, robotics, and games**.

PRACTICAL NO : 4
Knowledge Representation and Expert System

Title: Development of a Rule-Based Expert System using Propositional Logic, Forward Chaining and Backward Chaining

1. AIM

To study **knowledge representation using propositional logic** and develop a rule-based expert system using **Forward Chaining and Backward Chaining** for intelligent decision making.

2. OBJECTIVES

1. To understand the concept of knowledge representation.
2. To study propositional logic and logical rules.
3. To understand the structure of a rule-based expert system.
4. To study Forward Chaining and Backward Chaining mechanisms.
5. To understand how an expert system derives conclusions from given facts and rules.
6. To compare Forward Chaining and Backward Chaining.

3. THEORY

3.1 *Knowledge Representation*

Knowledge Representation (KR) is the process of representing facts, information, relationships, and rules in a form that can be processed by an Artificial Intelligence system.

Knowledge representation allows an AI system to:

- Store knowledge.
- Understand relationships between facts.
- Apply logical rules.
- Derive new information.
- Make decisions based on available knowledge.

Common knowledge representation techniques include:

1. Propositional Logic
2. Predicate Logic
3. Semantic Networks
4. Frames
5. Production Rules
6. Ontologies

4. PROPOSITIONAL LOGIC

Propositional Logic is a formal method of representing knowledge using propositions that can have one of two truth values:

- *True*
- **False**

A proposition is a statement that can be evaluated as true or false.

Examples

Let:

- **P:** It is raining.
- **Q:** The ground is wet.

A rule can be represented as:

P → Q

This means:

If it is raining, then the ground is wet.

If P is true, the rule allows us to conclude Q.

Common Logical Operators

Operator	Meaning	Example
¬	NOT	¬P
∧	AND	P ∧ Q
∨	OR	P ∨ Q
→	IMPLIES	P → Q
—	IF AND ONLY IF	P ↔ Q

5. RULE-BASED EXPERT SYSTEM

An **Expert System** is an AI-based system that uses stored knowledge and reasoning techniques to solve problems or make decisions in a specific domain.

A rule-based expert system generally consists of:

1. *Knowledge Base*

Contains:

- Facts
- Rules
    - Domain knowledge

Example:

Facts:

- It is raining.
- The ground is wet.

Rule:

- If it is raining, then the ground is wet.

2. *Inference Engine*

The inference engine applies rules to known facts and derives new conclusions.

3. *User Interface*

Allows the user to provide information and receive the decision or conclusion generated by the expert system.

6. EXAMPLE OF RULE-BASED EXPERT SYSTEM

Consider a simple **weather advisory expert system**.

Facts

- It is cloudy.
- It is raining.
- The ground is wet.

Rules

Rule 1:
If it is cloudy, then there is a possibility of rain.

Rule 2:
If it is raining, then the ground is wet.

Rule 3:
If it is raining and the ground is wet, then carry an umbrella.
Logical Representation

Cloudy → Possible_Rain
Raining → Wet_Ground
Raining ∧ Wet_Ground → Carry_Umbrella

The expert system uses these facts and rules to derive the final recommendation:

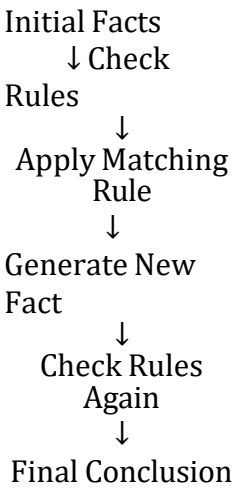
Carry Umbrella

7. FORWARD CHAINING

Forward Chaining is a data-driven reasoning technique.

It starts with the available facts and applies rules to derive new facts until the required conclusion is reached.

Working of Forward Chaining



Example

Given:

Fact 1: Raining
Fact 2: Wet_Ground

Rule:

Raining ∧ Wet_Ground → Carry_Umbrella

Since both conditions are true, the system derives:

Carry_Umbrella

Forward Chaining Algorithm

1. Start with the initial set of facts.
2. Check all available rules.
3. Identify rules whose conditions match the known facts.
4. Fire the matching rule.
5. Add the conclusion of the rule to the set of known facts.
6. Repeat the process with the newly generated facts.
7. Continue until the goal is reached or no more rules can be applied.
8. Display the final conclusion.
9. Stop.

8. BACKWARD CHAINING

Backward Chaining is a goal-driven reasoning technique.

Instead of starting from the available facts, it starts with a goal or conclusion and works backward to determine whether the required facts can be established.

Working of Backward Chaining

Goal
↓
Find Rule Supporting
Goal
↓
Check Rule Conditions
↓
Search for Supporting
Facts
↓ Verify
Facts
↓
Goal Established

Example

Suppose the goal is:

Carry_Umbrella

The system searches for a rule that can produce this conclusion:

Raining ∧ Wet_Ground → Carry_Umbrella

It then checks whether:

- Raining is true.
- Wet_Ground is true.

If both are established, the system concludes:

Carry_Umbrella Backward Chaining

Algorithm

1. Start with the desired goal.
2. Search for a rule whose conclusion matches the goal.
3. Identify the conditions required by that rule.
4. Check whether those conditions are already known facts.
5. If a condition is not known, treat it as a sub-goal.
6. Search for rules that can establish the sub-goal.
7. Continue until all required conditions are established or no suitable rule is found.
8. If all conditions are satisfied, conclude that the goal is true.
9. Otherwise, the goal cannot be established.
10. Stop.

9. FORWARD CHAINING VS BACKWARD CHAINING

Parameter	Forward Chaining	Backward Chaining
Approach	Data-driven	Goal-driven
Starts With	Facts	Goal
Direction	Facts → Conclusion	Goal → Facts
Main Focus	Deriving new facts	Proving a specific goal
Suitable For	Situations with many available facts	Situations with a specific goal
Search Process	Forward	Backward
Example	Weather monitoring	Diagnosis/query answering

10. ADVANTAGES

Advantages of Rule-Based Expert Systems

1. Easy to understand and implement.
2. Knowledge can be represented using simple rules.
3. Provides logical and explainable decisions.
4. Domain knowledge can be updated by adding or modifying rules.
5. Useful for decision-support applications.
6. Does not require a human expert to be available for every decision.

Advantages of Forward Chaining

1. Useful when many initial facts are available.
2. Can derive multiple conclusions from available information.
3. Suitable for monitoring and real-time decision-making.
4. Simple rule application mechanism.

Advantages of Backward Chaining

1. Efficient when there is a specific goal.
2. Avoids exploring unrelated rules.
3. Suitable for diagnosis and query-based systems.
4. Focuses only on rules relevant to the goal.

11. LIMITATIONS

Limitations of Rule-Based Expert Systems

1. Large numbers of rules can make the system complex.
2. Maintaining the knowledge base can become difficult.
3. The system cannot easily handle situations that are not covered by its rules.
4. Conflicting rules may produce inconsistent conclusions.
5. It does not automatically learn new knowledge unless a learning mechanism is added.

Limitations of Forward Chaining

1. May generate many unnecessary facts.
2. Can require more processing when the knowledge base is large.
3. May explore rules that are unrelated to the desired conclusion.

Limitations of Backward Chaining

1. Depends on having a clearly defined goal.
2. May require repeated searching for supporting rules.
3. Can become inefficient when there are many possible rules for a goal.
4. Performance depends on the structure of the rule base.

12. APPLICATIONS

Rule-based expert systems using logical reasoning are used in:

1. Medical diagnosis systems
2. Weather advisory systems
3. Fault detection systems
4. Financial decision support

5. Customer support systems
6. Computer troubleshooting
7. Industrial monitoring
8. Educational advisory systems
9. Security and intrusion detection
10. Agricultural decision-support systems

13. OBSERVATION

A rule-based expert system using **propositional logic** was studied. The working of **Forward Chaining and Backward Chaining** was understood for deriving conclusions from facts and rules.

14. CONCLUSION

In this practical, we studied **Knowledge Representation and Rule-Based Expert Systems** using propositional logic. We understood **Forward Chaining** as a data-driven approach and **Backward Chaining** as a goal-driven approach for intelligent decision making. Both techniques are useful for developing **intelligent and decision-support systems**.

Group -B

PRACTICAL NO. 5

DATA PREPROCESSING AND EXPLORATORY DATA ANALYSIS

Title: Data Preprocessing and Exploratory Data Analysis

Course Outcome: CO2

1. AIM

To perform data preprocessing and exploratory data analysis on a standard real-world dataset by applying data cleaning, missing value handling, feature scaling, encoding techniques, statistical analysis, and data visualization, and to divide the dataset into training and testing sets.

2. OBJECTIVES

After completing this practical, students will be able to:

1. Understand the importance of data preprocessing in machine learning.
2. Load and examine a standard real-world dataset.
3. Identify and clean inconsistent or unnecessary data.
4. Detect and handle missing values.
5. Apply suitable encoding techniques to categorical data.
6. Perform feature scaling on numerical attributes.
7. Perform basic statistical analysis of the dataset.
8. Visualize data distributions using histograms.
9. Identify the distribution and possible outliers using boxplots.
10. Split the preprocessed dataset into training and testing sets.

3. THEORY

3.1 Data Preprocessing

Data preprocessing is the process of converting raw data into a clean, consistent, and suitable format for machine learning.

Real-world datasets may contain missing values, duplicate records, categorical attributes, different numerical ranges, and inconsistent data. These issues can affect the performance and reliability of machine learning models.

The major steps involved in data preprocessing are: Data cleaning, Missing value handling, Data transformation, Encoding, Feature scaling, and Data splitting.

3.2 Data Cleaning

Data cleaning involves identifying and correcting errors or unwanted information in a

dataset.

Common data cleaning operations include:

1. Removing duplicate records.
2. Removing irrelevant attributes.
3. Correcting inconsistent values.
4. Checking data types.
5. Identifying invalid or abnormal values.
6. Checking the overall structure of the dataset.

Clean data provides a reliable foundation for further analysis and machine learning.

3.3 Missing Value Handling

A missing value represents unavailable or incomplete information in a dataset.

Missing values can be handled using different techniques:

- Mean – Replace missing numerical values with the mean.
- Median – Replace missing numerical values with the median.
- Mode – Replace missing categorical values with the most frequent value.
- Deletion – Remove records or attributes containing excessive missing values.

The appropriate technique depends on the nature and distribution of the data.

3.4 Encoding Techniques

Machine learning algorithms generally require numerical input. Therefore, categorical data must be converted into numerical form.

Label Encoding: Each category is assigned a numerical value.

Example: Female → 0, Male → 1.

One-Hot Encoding: Each category is represented using a separate binary attribute.

3.5 Feature Scaling

Different numerical features may have different ranges. Feature scaling transforms numerical attributes to a comparable scale.

Standardization:

$z = (x - \mu) / \sigma$

where x is the original value, μ is the mean, and σ is the standard deviation.

Min-Max Scaling:

$x' = (x - xmin) / (xmax - xmin)$

Feature scaling is particularly important for algorithms that are sensitive to the magnitude of features.

3.6 Statistical Analysis

Statistical analysis helps in understanding the characteristics of a dataset.

Important statistical measures include Mean, Median, Mode, Minimum, Maximum, Variance, Standard deviation, and Quartiles.

Descriptive statistics provide a summarized view of numerical features and help identify unusual patterns in the data.

3.7 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset using statistical methods and graphical techniques.

EDA helps to:

- Understand the distribution of data.
- Identify patterns and relationships.
- Detect outliers.
- Identify unusual observations.
- Understand feature characteristics.
- Support further machine learning analysis.

3.8 Histogram

A histogram is a graphical representation of the frequency distribution of numerical data.

It divides the data into intervals called bins and displays the number of observations within each interval.

Histograms can be used to understand whether data is normally distributed, positively skewed, negatively skewed, or uniformly distributed.

3.9 Boxplot

A boxplot represents the distribution of numerical data using Minimum, First quartile (Q1), Median, Third quartile (Q3), and Maximum.

It is also useful for identifying potential outliers.

The interquartile range is given by:

IQR = Q3 − Q1

Potential outliers can be identified using:

Lower Limit = Q1 − 1.5(IQR)

Upper Limit = Q3 + 1.5(IQR)

4. DATASET

For this practical, a standard Titanic dataset can be used.

The Titanic dataset contains information about passengers and their survival status.

Important attributes include:

- survived – Survival status of the passenger
- pclass – Passenger class
- sex – Gender of passenger
- age – Age of passenger
- sibsp – Number of siblings/spouses aboard
- parch – Number of parents/children aboard
- fare – Passenger fare
- embarked – Port of embarkation

The dataset is suitable for this practical because it contains both numerical and categorical attributes, as well as missing values that can be used to demonstrate preprocessing techniques.

5. ALGORITHM

Algorithm: Data Preprocessing and Exploratory Data Analysis

Step 1: Start.

Step 2: Select and download a standard dataset such as the Titanic dataset.

Step 3: Load the dataset and examine its structure, attributes, and data types.

Step 4: Check the dataset for duplicate, irrelevant, inconsistent, or invalid records.

Step 5: Identify the missing values present in the dataset.

Step 6: Handle missing numerical values using an appropriate method such as mean or

median.

Step 7: Handle missing categorical values using an appropriate method such as mode.

Step 8: Remove unnecessary or irrelevant attributes.

Step 9: Perform statistical analysis on the numerical features.

Step 10: Generate histograms to study the distribution of numerical attributes.

Step 11: Generate boxplots to understand data distribution and identify potential outliers.

Step 12: Identify categorical attributes in the dataset.

Step 13: Apply an appropriate encoding technique to convert categorical attributes into numerical form.

Step 14: Apply feature scaling to the required numerical attributes.

Step 15: Separate the input features from the target variable.

Step 16: Divide the processed dataset into training and testing sets.

Step 17: Examine the preprocessed dataset to ensure that it is ready for machine learning.

Step 18: Stop.

6. OBSERVATION

1. The dataset contains both numerical and categorical attributes.
2. Data cleaning is required before applying machine learning algorithms.
3. Missing values need to be handled appropriately to avoid incomplete data.
4. Categorical attributes need to be converted into numerical representation.
5. Numerical features may have different ranges and therefore require scaling.
6. Statistical analysis provides useful information about the characteristics of the dataset.
7. Histograms help in understanding the distribution of numerical features.
8. Boxplots help in identifying the spread of data and potential outliers.
9. Data preprocessing improves the quality and consistency of the dataset.
10. The final dataset can be divided into training and testing sets for subsequent machine learning tasks.

7. APPLICATIONS OF DATA PREPROCESSING AND EDA

1. Healthcare – disease prediction and patient data analysis.
2. Banking and Finance – fraud detection and credit-risk analysis.
3. Education – student performance and dropout prediction.
4. E-Commerce – customer behavior analysis and recommendation systems.
5. Transportation – passenger and traffic analysis.
6. Marketing – customer segmentation and campaign analysis.
7. Manufacturing – quality analysis and predictive maintenance.

8. Machine Learning – preparation of datasets for classification, regression, and clustering.

8. CONCLUSION

In this practical, data preprocessing and exploratory data analysis were performed on a standard real-world dataset. The dataset was cleaned, missing values were handled, categorical attributes were encoded, and numerical features were scaled. Statistical analysis and visualization techniques such as histograms and boxplots were used to understand the characteristics and distribution of the data. Finally, the processed dataset was divided into training and testing sets, making it suitable for further machine learning applications.

PRACTICAL NO. 6

REGRESSION TECHNIQUES

Title: Regression Techniques

Course Outcomes: CO2, CO3

1. AIM

To implement and study Linear Regression and Polynomial Regression using a suitable real-world dataset and evaluate and compare their performance using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score.

2. OBJECTIVES

After completing this practical, students will be able to:

1. Understand the concept of regression in machine learning.
2. Understand the working principle of Linear Regression.
3. Understand the concept of Polynomial Regression.
4. Apply Linear Regression to a real-world dataset.
5. Apply Polynomial Regression to a real-world dataset.
6. Evaluate regression models using MSE, MAE, and R^2 Score.
7. Compare the performance of Linear and Polynomial Regression models.
8. Identify the suitability of different regression techniques for prediction problems.

3. THEORY

3.1 Regression

Regression is a supervised machine learning technique used to predict a continuous numerical output based on one or more input variables.

Examples of regression applications include:

- House price prediction
- Temperature prediction
- Sales forecasting
- Stock price prediction
- Salary prediction
- Demand forecasting

In regression, the relationship between independent variables and a dependent variable is studied to make predictions.

3.2 Linear Regression

Linear Regression is a supervised learning algorithm that models the relationship between input variables and a continuous target variable using a linear equation.

For simple linear regression, the equation is:

$y = b_0 + b_1x$

where:

- y = predicted output
- x = input feature
- b_0 = intercept
- b_1 = regression coefficient

The objective is to find the best-fitting line that minimizes the difference between actual and predicted values.

3.3 Multiple Linear Regression

When more than one independent variable is used for prediction, it is called Multiple Linear Regression.

The general equation is:

$y = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n$

where $x_1, x_2, ..., x_n$ represent multiple input features.

3.4 Polynomial Regression

Polynomial Regression is an extension of Linear Regression that models a nonlinear relationship between the independent and dependent variables.

A polynomial regression equation of degree 2 can be represented as:

$y = b_0 + b_1x + b_2x^2$

For degree 3:

$y = b_0 + b_1x + b_2x^2 + b_3x^3$

Polynomial Regression can provide a better fit when the relationship between input and output is nonlinear.

3.5 Linear Regression vs Polynomial Regression

Feature	Linear Regression	Polynomial Regression
Relationship	Linear	Nonlinear
Model	Straight line	Polynomial curve

Complexity	Low	Higher
Number of features	Original features	Includes polynomial terms
Risk of overfitting	Relatively low	Higher at large polynomial degrees
Suitable for	Linear relationships	Curved/nonlinear relationships

4. DATASET

For this practical, the California Housing dataset can be used as a standard real-world dataset.

The dataset contains information related to housing characteristics and their corresponding median house values in different districts of California.

Important attributes include:

Attribute	Description
MedInc	Median income
HouseAge	Median house age
AveRooms	Average number of rooms
AveBedrms	Average number of bedrooms
Population	Population of the district
AveOccup	Average household occupancy
Latitude	Geographic latitude
Longitude	Geographic longitude
MedHouseVal	Median house value

The MedHouseVal attribute is used as the target variable for house-value prediction.

5. ALGORITHM

Algorithm: Linear Regression and Polynomial Regression

Step 1: Start.

Step 2: Select and download a suitable regression dataset such as the California Housing dataset.

Step 3: Load and examine the dataset.

Step 4: Identify the input features and target variable.

Step 5: Check the dataset for missing or invalid values.

Step 6: Perform necessary data preprocessing.

Step 7: Divide the dataset into training and testing sets.

Step 8: Train a Linear Regression model using the training dataset.

Step 9: Use the Linear Regression model to predict the target values for the testing dataset.

Step 10: Calculate the MSE, MAE, and R^2 Score for the Linear Regression model.

Step 11: Transform the input features into polynomial features.

Step 12: Train a Polynomial Regression model using the transformed training data.

Step 13: Predict the target values using the Polynomial Regression model.

Step 14: Calculate the MSE, MAE, and R^2 Score for the Polynomial Regression model.

Step 15: Compare the performance of both regression techniques.

Step 16: Identify the model that provides better prediction performance based on the evaluation metrics.

Step 17: Stop.

6. MODEL EVALUATION METRICS

6.1 Mean Squared Error (MSE)

Mean Squared Error measures the average squared difference between actual and predicted values.

$$MSE = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$$

where:

- y_i = actual value
- $\hat{y}_i$ = predicted value
- n = number of observations

Lower MSE indicates better model performance.

6.2 Mean Absolute Error (MAE)

Mean Absolute Error measures the average absolute difference between actual and predicted values.

$$MAE = (1/n)\ \Sigma|y_i - \hat{y}_i|$$

Lower MAE indicates better prediction accuracy.

6.3 R^2 Score

R^2 Score, also known as the coefficient of determination, measures how well the model explains the variation in the target variable.

$R^2 = 1 - SSres/SStot$

where:

- SSres = residual sum of squares
- SStot = total sum of squares

The value of R^2 is generally interpreted as:

- R^2 closer to 1: Better model performance
- R^2 closer to 0: Poor explanatory capability

Therefore, a model with lower MSE and MAE and higher R^2 Score is generally preferred.

7. OBSERVATION

1. Linear Regression models the relationship between input and target variables using a linear relationship.
2. Polynomial Regression extends the regression approach by introducing polynomial terms.
3. Linear Regression is simpler and computationally less complex.
4. Polynomial Regression can capture nonlinear relationships that a straight-line model cannot represent.
5. Increasing the polynomial degree can increase model complexity.
6. A very high polynomial degree may lead to overfitting.
7. MSE and MAE are error-based evaluation metrics, where lower values indicate better performance.
8. R^2 Score is a goodness-of-fit measure, where a higher value generally indicates better performance.
9. The performance of Linear and Polynomial Regression can be compared using the three evaluation metrics.
10. The more suitable model depends on the characteristics and relationship present in the dataset.

8. COMPARISON OF REGRESSION TECHNIQUES

Evaluation Criterion	Linear Regression	Polynomial Regression
Relationship captured	Linear	Nonlinear
Model complexity	Low	Higher
Training complexity	Low	Higher
Flexibility	Limited	Higher
MSE	Compare calculated value	Compare calculated value
MAE	Compare calculated value	Compare calculated value
R^2 Score	Compare calculated value	Compare calculated value
Overfitting risk	Lower	Higher
Suitable for	Approximately linear relationships	Nonlinear relationships

The final model should be selected based on the evaluation results obtained from MSE, MAE, and R^2 Score rather than assuming that one technique will always outperform the other.

9. APPLICATIONS

1. House Price Prediction – predicting property prices based on different characteristics.
2. Sales Forecasting – estimating future sales.
3. Demand Prediction – predicting product demand.
4. Financial Analysis – predicting financial values and trends.
5. Healthcare – predicting continuous medical measurements.
6. Weather Forecasting – predicting temperature and other continuous variables.
7. Business Analytics – estimating revenue and expenditure.
8. Energy Management – predicting energy consumption.
9. Real Estate Analytics – estimating property values.
10. Risk Analysis – estimating continuous risk-related measures.

10. CONCLUSION

In this practical, Linear Regression and Polynomial Regression were studied and applied to a suitable real-world dataset. The models were evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score. The performance of both techniques was compared based on their prediction errors and goodness of fit. Linear Regression is suitable for approximately linear relationships, while Polynomial Regression can model more complex nonlinear relationships. Thus, the selection of the regression technique depends on the underlying characteristics of the dataset and the evaluation results.

PRACTICAL NO. 7

CLASSIFICATION USING LOGISTIC REGRESSION AND K-NEAREST NEIGHBORS

Title: Classification using Logistic Regression and K-Nearest Neighbors (KNN)

Course Outcomes: CO2, CO3

1. AIM

To perform data preprocessing and implement Logistic Regression and K-Nearest Neighbors (KNN) classification techniques, evaluate their performance using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix, and compare the results obtained from both models.

2. OBJECTIVES

After completing this practical, students will be able to:

1. Understand the concept of classification in machine learning.
2. Understand the working principle of Logistic Regression.
3. Understand the working principle of K-Nearest Neighbors.
4. Perform preprocessing on a classification dataset.
5. Divide the dataset into training and testing sets.
6. Implement Logistic Regression for classification.
7. Implement K-Nearest Neighbors for classification.
8. Evaluate classification models using different performance metrics.
9. Interpret a confusion matrix.
10. Compare the performance of Logistic Regression and KNN.

3. THEORY

3.1 Classification

Classification is a supervised machine learning technique used to predict a categorical class or label based on one or more input features.

Examples of classification problems include:

- Disease detection
- Email spam detection
- Student result classification
- Fraud detection
- Customer churn prediction
- Image classification

Unlike regression, where the output is continuous, classification produces discrete class labels.

3.2 Logistic Regression

Logistic Regression is a supervised machine learning algorithm primarily used for classification problems.

Although it contains the word "regression", it is used to predict the probability that an observation belongs to a particular class.

For binary classification, the logistic or sigmoid function is used:

$P(y=1) = 1 / (1 + e^{-z})$

where:

$z = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$

The sigmoid function produces a value between 0 and 1, which can be interpreted as a probability.

A classification threshold is then used to determine the final class.

3.3 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a supervised machine learning algorithm used for classification and regression.

In classification, KNN assigns a class to a new observation based on the classes of its nearest neighboring observations.

The basic procedure is:

1. Select the value of K.
2. Calculate the distance between the new observation and training observations.
3. Identify the K nearest observations.
4. Count the classes of the nearest neighbors.
5. Assign the class having the highest number of votes.

3.4 Distance in KNN

KNN commonly uses Euclidean distance.

For two points $P(x_1,y_1)$ and $Q(x_2,y_2)$, Euclidean distance is:

$d(P,Q) = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$

For multiple features, the distance is calculated across all feature dimensions.

3.5 Importance of Feature Scaling in KNN

Feature scaling is particularly important for KNN because the algorithm calculates distances between observations.

If one feature has a much larger numerical range than another, it may dominate the distance calculation.

Therefore, techniques such as Standardization or Min-Max Scaling are commonly applied before implementing KNN.

4. DATASET

For this practical, a standard classification dataset such as the Breast Cancer Wisconsin dataset can be used.

The dataset contains numerical measurements of cell characteristics and a target class indicating whether the tumor is malignant or benign.

Important characteristics include:

Feature Category	Description
Radius	Mean radius of the cell
Texture	Texture-related measurement
Perimeter	Mean perimeter
Area	Mean cell area
Smoothness	Smoothness measurement
Compactness	Compactness measurement
Concavity	Concavity measurement
Symmetry	Symmetry measurement
Fractal Dimension	Fractal dimension measurement
Target	Tumor classification

The dataset is suitable for comparing Logistic Regression and KNN because it is a binary classification problem with numerical features.

5. ALGORITHM

Algorithm: Logistic Regression and K-Nearest Neighbors

Step 1: Start.

Step 2: Select and load a suitable classification dataset.

Step 3: Examine the dataset and identify its features and target variable.

Step 4: Check for missing, duplicate, or invalid values.

Step 5: Perform the required data preprocessing.

Step 6: Separate the input features and target class.

Step 7: Split the dataset into training and testing sets.

Step 8: Apply feature scaling to the input features.

Step 9: Train the Logistic Regression model using the training dataset.

Step 10: Predict the class labels for the testing dataset using Logistic Regression.

Step 11: Calculate Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

Step 12: Select an appropriate value of K for KNN.

Step 13: Train the KNN classifier using the training dataset.

Step 14: Predict the class labels for the testing dataset using KNN.

Step 15: Calculate Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

Step 16: Compare the evaluation results of Logistic Regression and KNN.

Step 17: Identify the better-performing model based on the evaluation metrics.

Step 18: Stop.

6. MODEL EVALUATION METRICS

6.1 Accuracy

Accuracy represents the proportion of correctly classified observations out of the total observations.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

where TP = True Positive, TN = True Negative, FP = False Positive, and FN = False Negative.

Higher accuracy generally indicates better classification performance.

6.2 Precision

Precision represents the proportion of correctly predicted positive observations among all observations predicted as positive.

Precision = TP / (TP + FP)

High precision indicates fewer false-positive predictions.

6.3 Recall

Recall represents the proportion of actual positive observations that are correctly identified by the model.

Recall = TP / (TP + FN)

High recall indicates fewer false-negative predictions.

6.4 F1-Score

F1-Score is the harmonic mean of Precision and Recall.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

F1-Score is particularly useful when both precision and recall are important.

6.5 Confusion Matrix

A confusion matrix is a table used to evaluate classification predictions.

For binary classification, it contains:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

The confusion matrix helps identify the types of classification errors made by the model.

7. COMPARISON OF LOGISTIC REGRESSION AND KNN

Criterion	Logistic Regression	KNN
Type	Supervised classification	Supervised classification
Basic approach	Probability-based classification	Neighbor-based classification
Training complexity	Relatively low	Very low
Prediction complexity	Low	Higher
Feature scaling	Useful	Important
Interpretability	Relatively high	Lower
Main parameter	Regularization/model coefficients	Value of K
Nonlinear patterns	Limited without transformation	Can capture complex boundaries
Sensitivity to irrelevant features	Relatively lower	Can be higher
Computational requirement	Generally lower during prediction	Higher during prediction

The final comparison should be based on the actual Accuracy, Precision, Recall, F1-Score, and Confusion Matrix obtained after implementing both models.

8. OBSERVATION

1. Classification algorithms predict discrete class labels.
2. Logistic Regression uses probabilities to classify observations into different classes.
3. KNN classifies an observation based on the classes of its nearest neighbors.
4. Feature scaling is particularly important for KNN because it uses distance calculations.
5. The value of K influences the performance of the KNN classifier.
6. Accuracy provides an overall measure of correct predictions.
7. Precision measures the correctness of positive predictions.

8. Recall measures the ability to identify actual positive observations.
9. F1-Score provides a balance between Precision and Recall.
10. The confusion matrix provides detailed information about correct and incorrect classifications.
11. The better model should be selected based on the evaluation metrics and the requirements of the classification problem.

9. APPLICATIONS

1. Healthcare – disease and medical condition classification.
2. Banking – credit approval and customer risk classification.
3. Fraud Detection – identifying fraudulent transactions.
4. Email Classification – spam and non-spam classification.
5. Education – student performance classification.
6. Marketing – customer response prediction.
7. Customer Analytics – customer churn classification.
8. Image Recognition – classification of objects and patterns.
9. Cybersecurity – classification of normal and malicious activities.
10. Manufacturing – product quality classification.

10. CONCLUSION

In this practical, Logistic Regression and K-Nearest Neighbors (KNN) classification techniques were studied and implemented on a suitable classification dataset. The models were evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The evaluation metrics provide different perspectives on classification performance, while the confusion matrix helps analyze correct and incorrect predictions. Finally, the results of Logistic Regression and KNN were compared to determine the more suitable classification technique for the selected dataset.

PRACTICAL NO. 8

CLASSIFICATION USING DECISION TREE AND NAÏVE BAYES

Title: Classification using Decision Tree and Naïve Bayes

Course Outcomes: CO2, CO3

1. AIM

To implement Decision Tree and Naïve Bayes classifiers on a suitable classification dataset, evaluate their performance using standard classification metrics, and compare the results to identify the most suitable model for the selected dataset.

2. OBJECTIVES

After completing this practical, students will be able to:

1. Understand the concept of classification in machine learning.
2. Understand the working principle of the Decision Tree classifier.
3. Understand the working principle of the Naïve Bayes classifier.
4. Perform necessary preprocessing on a classification dataset.
5. Train a Decision Tree classification model.
6. Train a Naïve Bayes classification model.
7. Evaluate classification models using standard performance metrics.
8. Analyze the strengths and limitations of both classifiers.
9. Compare the performance of Decision Tree and Naïve Bayes.
10. Identify the most suitable classifier for the selected dataset.

3. THEORY

3.1 Classification

Classification is a supervised machine learning technique used to assign observations to predefined classes or categories.

The model learns patterns from labelled training data and uses these patterns to predict the class of new observations.

Examples include:

- Disease classification
- Spam detection
- Fraud detection
- Customer churn prediction
- Student performance classification
- Image classification

3.2 Decision Tree

A Decision Tree is a supervised machine learning algorithm that represents decisions in the form of a tree structure.

A Decision Tree consists of:

- Root Node – represents the first decision or feature.
- Internal Nodes – represent decision conditions.
- Branches – represent outcomes of decisions.
- Leaf Nodes – represent the final class predictions.

The algorithm recursively divides the dataset based on selected features until a suitable stopping condition is reached.

3.3 Working of Decision Tree

The basic working process of a Decision Tree is:

1. Start with the complete training dataset.
2. Select the most suitable feature for splitting.
3. Divide the dataset according to the selected feature.
4. Repeat the splitting process for the resulting subsets.
5. Continue until a stopping condition is reached.
6. Assign class labels to the leaf nodes.
7. Use the resulting tree to classify new observations.

The selection of a suitable feature can be based on measures such as Gini Impurity or Entropy.

3.4 Entropy

Entropy measures the impurity or uncertainty in a dataset.

$\text{Entropy}(S) = -\Sigma\, p_i \log_2(p_i)$

where:

- S = dataset
- p_i = proportion of observations belonging to class i

Lower entropy indicates a purer set of observations.

3.5 Gini Impurity

Gini Impurity is another criterion used to measure the impurity of a node.

$\text{Gini} = 1 - \Sigma\, p_i^2$

A lower Gini value represents greater purity of the node.

3.6 Naïve Bayes

Naïve Bayes is a probabilistic supervised machine learning classification algorithm

based on Bayes' theorem.

Bayes' theorem is given by:

$P(A|B) = P(B|A)P(A) / P(B)$

where:

- $P(A|B)$ = posterior probability
- $P(B|A)$ = likelihood
- $P(A)$ = prior probability
- $P(B)$ = evidence

3.7 Naïve Bayes Assumption

Naïve Bayes assumes that the input features are conditionally independent given the class.

Although this assumption may not always be completely true in real-world datasets, Naïve Bayes can still provide effective classification performance.

The classifier calculates the probability of each possible class and assigns the observation to the class with the highest posterior probability.

3.8 Types of Naïve Bayes

Type	Common Application
Gaussian Naïve Bayes	Continuous numerical data
Multinomial Naïve Bayes	Text and frequency-based data
Bernoulli Naïve Bayes	Binary features

For numerical datasets, Gaussian Naïve Bayes is commonly used.

4. DATASET

For this practical, a standard classification dataset such as the Iris dataset can be used.

The Iris dataset contains measurements of iris flowers belonging to three different species.

Important features include:

Feature	Description
Sepal Length	Length of the sepal
Sepal Width	Width of the sepal
Petal Length	Length of the petal
Petal Width	Width of the petal
Species	Target class

The target consists of three classes:

1. Setosa
2. Versicolor

3. Virginic

The Iris dataset is suitable for comparing Decision Tree and Naïve Bayes because it contains numerical features and multiple classification classes.

5. ALGORITHM

Algorithm: Decision Tree and Naïve Bayes Classification

Step 1: Start.

Step 2: Select and load a suitable classification dataset such as the Iris dataset.

Step 3: Examine the dataset and identify input features and the target variable.

Step 4: Check the dataset for missing, duplicate, or invalid values.

Step 5: Perform the required data preprocessing.

Step 6: Separate the input features from the target class.

Step 7: Divide the dataset into training and testing sets.

Step 8: Train the Decision Tree classifier using the training dataset.

Step 9: Use the Decision Tree model to predict the classes of the testing dataset.

Step 10: Evaluate the Decision Tree model using standard classification metrics.

Step 11: Train the Naïve Bayes classifier using the training dataset.

Step 12: Use the Naïve Bayes model to predict the classes of the testing dataset.

Step 13: Evaluate the Naïve Bayes model using standard classification metrics.

Step 14: Compare the evaluation results of both classifiers.

Step 15: Identify the classifier that provides better performance for the selected dataset.

Step 16: Stop.

6. MODEL EVALUATION METRICS

6.1 Accuracy

Accuracy represents the proportion of correctly classified observations among all observations.

Accuracy = Correct Predictions / Total Predictions

Higher accuracy generally indicates better classification performance.

6.2 Precision

Precision measures how many of the observations predicted as a particular class actually belong to that class.

Precision = TP / (TP + FP)

Higher precision indicates fewer false-positive predictions.

6.3 Recall

Recall measures how effectively the model identifies observations that actually belong to a

particular class.

Recall = TP / (TP + FN)

Higher recall indicates fewer false-negative predictions.

6.4 F1-Score

F1-Score combines Precision and Recall into a single metric.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

A higher F1-Score indicates a better balance between precision and recall.

6.5 Confusion Matrix

A confusion matrix provides a detailed representation of classification results.

For a multi-class classification problem such as Iris, each row represents an actual class and each column represents a predicted class.

The diagonal elements represent correctly classified observations, while the off-diagonal elements represent misclassifications.

7. COMPARISON OF DECISION TREE AND NAÏVE BAYES

Criterion	Decision Tree	Naïve Bayes
Type	Supervised classification	Probabilistic classification
Basic principle	Decision rules	Bayes' theorem
Model structure	Tree structure	Probability-based
Feature independence assumption	Not required	Assumed
Interpretability	High	Moderate
Numerical data	Suitable	Suitable
Categorical data	Suitable	Suitable depending on variant
Training speed	Generally fast	Very fast
Model complexity	Can become high	Relatively low
Overfitting	Possible without pruning	Generally lower
Main advantage	Easy-to-understand decision rules	Simple and computationally efficient
Main limitation	Can overfit training data	Independence

		assumption may be unrealistic

The final model should be selected based on the actual evaluation results obtained from the selected dataset.

8. OBSERVATION

1. Decision Tree uses a tree-based structure to make classification decisions.
2. Naïve Bayes uses probability calculations based on Bayes' theorem.
3. Decision Tree does not require the assumption of independent features.
4. Naïve Bayes assumes conditional independence among features.
5. Decision Trees are generally easy to interpret and visualize.
6. Naïve Bayes is computationally efficient and can perform well with relatively simple models.
7. Accuracy provides an overall measure of correct classifications.
8. Precision, Recall, and F1-Score provide additional information about classification performance.
9. The confusion matrix helps identify correctly classified and misclassified observations for each class.
10. The most suitable classifier should be selected based on the evaluation metrics and the characteristics of the dataset.

9. APPLICATIONS

1. Healthcare – disease and medical condition classification.
2. Banking – credit-risk and loan approval classification.
3. Email Filtering – spam and non-spam classification.
4. Text Classification – document and sentiment classification.
5. Fraud Detection – identifying suspicious transactions.
6. Education – student performance classification.
7. Customer Analytics – customer behavior and churn classification.
8. Cybersecurity – classification of normal and malicious activities.
9. Recommendation Systems – classification of user preferences.
10. Business Analytics – customer and product classification.

10. CONCLUSION

In this practical, Decision Tree and Naïve Bayes classification techniques were studied and applied to a suitable classification dataset. Both classifiers were evaluated using standard performance metrics such as Accuracy, Precision, Recall, F1-Score, and

Confusion Matrix. The results of both models were compared to understand their strengths, limitations, and classification performance. The most suitable model can be selected based on the evaluation results and the characteristics of the selected dataset.

Group C

PRACTICAL NO. 9

Clustering Techniques (CO4)

1. AIM

To implement and analyze K-Means Clustering and Hierarchical Clustering techniques using a real-world dataset, and determine the optimal number of clusters using the Elbow Method and Dendrogram.

2. OBJECTIVES

- To understand the concept of clustering in unsupervised learning.
- To study the working principle of K-Means Clustering.
- To understand the concept and working of Hierarchical Clustering.
- To determine the optimal number of clusters using the Elbow Method.
- To determine the appropriate number of clusters using a Dendrogram.
- To analyze the characteristics of the generated clusters.
- To compare K-Means and Hierarchical Clustering techniques.

3. THEORY

3.1 Clustering

Clustering is an unsupervised machine learning technique used to group similar data objects into clusters. Data points within the same cluster are expected to have greater similarity, while data points belonging to different clusters should have relatively less similarity.

Unlike supervised learning, clustering does not require predefined target labels.

Common applications of clustering include:

- Customer segmentation
- Market analysis
- Image segmentation
- Document classification
- Anomaly detection
- Social network analysis
- Medical data analysis

3.2 K-Means Clustering

K-Means is a partition-based clustering algorithm that divides the dataset into K predefined clusters.

Each cluster is represented by a centroid, which is the mean position of the data points assigned to that cluster.

The objective of K-Means is to minimize the Within-Cluster Sum of Squares (WCSS).

$WCSS = \Sigma(i=1 \text{ to } K) \Sigma(x \in Ci) ||x - \mu i||^2$

Where: K = number of clusters; Ci = set of observations belonging to cluster i; x = data point; μi = centroid of cluster i.

Working of K-Means:

- Select the number of clusters K.
- Initialize K cluster centroids.
- Assign each data point to the nearest centroid.
- Recalculate the centroid of each cluster.
- Reassign the data points based on the updated centroids.
- Repeat the process until the centroids stabilize or the stopping condition is reached.

3.3 Elbow Method

The Elbow Method is used to determine a suitable value of K for K-Means clustering. Different values of K are tested and the corresponding WCSS values are calculated. The WCSS generally decreases as the number of clusters increases.

The point where the decrease in WCSS becomes considerably smaller is known as the elbow point. This point is generally considered a suitable value for the number of clusters.

3.4 Hierarchical Clustering

Hierarchical Clustering is an unsupervised learning technique that creates a hierarchy of clusters.

It can be performed using two approaches:

- Agglomerative Hierarchical Clustering – starts with individual data points as separate clusters and repeatedly merges the closest clusters.
- Divisive Hierarchical Clustering – starts with all data points in one cluster and progressively divides them into smaller clusters.

Agglomerative clustering is commonly used because of its simple bottom-up approach.

3.5 Dendrogram

A Dendrogram is a tree-like diagram used to represent the hierarchy of clusters formed during hierarchical clustering.

It helps determine the appropriate number of clusters by observing where a horizontal

line can be drawn through the dendrogram without crossing many vertical branches. The number of major groups formed by the selected cut provides an estimate of the appropriate number of clusters.

3.6 Distance Measures

Clustering algorithms use distance or similarity measures to determine how close data points are.

Euclidean Distance: $d(x,y) = \sqrt{[\Sigma(i=1 \text{ to } n)(xi - yi)^2]}$

Where x and y represent two observations.

4. DATASET

A suitable real-world dataset such as the Mall Customers dataset can be used for clustering analysis.

Attribute	Description
Customer ID	Unique identification number of customer
Gender	Gender of customer
Age	Age of customer
Annual Income	Annual income of customer
Spending Score	Customer spending score

For clustering, relevant numerical attributes such as Annual Income and Spending Score can be selected.

Since clustering is an unsupervised technique, a predefined target class is not required.

5. ALGORITHM

A. K-Means Clustering

- Start.
- Load the real-world dataset.
- Inspect the dataset structure and attributes.
- Check for missing, duplicate, and invalid values.
- Select relevant numerical features for clustering.
- Perform appropriate preprocessing and feature scaling.
- Calculate WCSS for different values of K.
- Plot the WCSS values against the number of clusters.
- Identify the elbow point to determine a suitable value of K.
- Initialize the K-Means algorithm with the selected number of clusters.
- Assign each observation to its nearest centroid.

- Recalculate the cluster centroids.
- Repeat the assignment and centroid update process until convergence.
- Analyze the resulting clusters.
- Record the cluster characteristics.

B. Hierarchical Clustering

- Start.
- Load the same preprocessed dataset.
- Select relevant features for clustering.
- Calculate the distance between observations.
- Treat each observation as an individual cluster.
- Identify the closest clusters.
- Merge the closest clusters.
- Repeat the merging process until a hierarchical structure is obtained.
- Generate the Dendrogram.
- Examine the Dendrogram to identify an appropriate number of clusters.
- Divide the hierarchy according to the selected cluster level.
- Analyze the generated clusters.
- Record the characteristics of each cluster.

6. CLUSTER ANALYSIS

After generating the clusters, the characteristics of each cluster should be examined.

- Number of observations in each cluster.
- Average annual income of each cluster.
- Average spending score of each cluster.
- Similarity between observations within a cluster.
- Differences between different clusters.
- Identification of high-income and high-spending customers.
- Identification of low-income and low-spending customers.
- Identification of other meaningful customer segments.

The resulting clusters can be interpreted according to the characteristics of the observations assigned to them.

7. COMPARISON OF CLUSTERING TECHNIQUES

Parameter	K-Means Clustering	Hierarchical Clustering
Type	Partition-based	Hierarchical
Number of clusters	Usually specified in advance	Can be determined using Dendrogram
Cluster representation	Centroids	Hierarchical structure
Main technique	Iterative centroid optimization	Repeated merging or splitting
Visualization	Cluster plot	Dendrogram
Large datasets	Generally efficient	Can be computationally expensive
Cluster interpretation	Based on cluster centroids	Based on hierarchy and linkage
Main limitation	Requires suitable K	Sensitive to distance and linkage method

8. OBSERVATION

- Clustering successfully groups observations having similar characteristics.
- The Elbow Method provides a graphical approach for selecting a suitable number of clusters for K-Means.
- The Dendrogram provides a hierarchical representation of the relationships among observations.
- Feature scaling is important when features have different ranges.
- K-Means is suitable when a relatively compact partition of the data is required.
- Hierarchical clustering is useful when the relationship between clusters and their hierarchical structure needs to be studied.
- The generated clusters can provide meaningful segments for further analysis and decision-making.

9. APPLICATIONS OF CLUSTERING

- Customer Segmentation: Grouping customers according to purchasing behavior.
- Marketing Analysis: Identifying different market segments.
- Healthcare: Grouping patients according to similar characteristics.
- Image Processing: Segmenting images into meaningful regions.

- Document Analysis: Grouping similar documents or articles.
- Fraud Detection: Identifying unusual groups or patterns in transactions.
- Social Network Analysis: Finding groups of users with similar behavior.
- Recommendation Systems: Grouping users or products based on similarities.
- Geographical Analysis: Identifying regions with similar characteristics.

10. CONCLUSION

K-Means and Hierarchical Clustering are important unsupervised machine learning techniques for discovering hidden patterns and natural groups within datasets. The Elbow Method helps determine a suitable number of clusters for K-Means, while the Dendrogram provides a visual representation for selecting clusters in hierarchical clustering. By applying these techniques to a real-world dataset, meaningful groups can be identified and analyzed based on their characteristics.

PRACTICAL NO. 10

Dimensionality Reduction using Principal Component Analysis (PCA) (CO4)

1. AIM

To apply Principal Component Analysis (PCA) on a high-dimensional dataset, reduce the number of features, visualize the reduced-dimensional data, and analyze the impact of dimensionality reduction on classification performance.

2. OBJECTIVES

- To understand the concept of dimensionality reduction.
- To study the working principle of Principal Component Analysis.
- To apply PCA to a high-dimensional dataset.
- To compare the number of features before and after PCA.
- To visualize data in reduced dimensions.
- To evaluate the impact of PCA on classification performance.
- To compare classification performance before and after dimensionality reduction.

3. THEORY

3.1 Dimensionality Reduction

Dimensionality reduction is the process of reducing the number of input features while retaining the important information present in the original dataset.

High-dimensional datasets may contain redundant, correlated, or less important features. A large number of features can increase computational complexity and may lead to the curse of dimensionality.

Dimensionality reduction can help to:

- Reduce computational complexity.
- Remove redundant information.
- Improve visualization.
- Reduce storage requirements.
- Minimize the effect of noisy features.
- Improve the efficiency of machine learning models.

3.2 Principal Component Analysis (PCA)

Principal Component Analysis is a widely used linear dimensionality reduction technique. PCA transforms the original correlated features into a smaller set of new variables called Principal Components.

Each principal component is a linear combination of the original features.

The first principal component captures the maximum possible variance in the data. The second principal component captures the maximum remaining variance while being orthogonal to the first component. This process continues for subsequent components.

3.3 Working of PCA

- Standardize the numerical features.
- Calculate the covariance matrix.
- Calculate the eigenvalues and eigenvectors of the covariance matrix.
- Rank the principal components according to their eigenvalues.
- Select the required number of principal components.
- Transform the original dataset into the reduced feature space.

3.4 Explained Variance

Explained Variance Ratio indicates how much information or variance from the original dataset is retained by each principal component.

The cumulative explained variance can be used to determine an appropriate number of components.

For example, if the first few principal components retain a large percentage of the total variance, the remaining components may be removed with relatively small information loss.

4. DATASET

A high-dimensional dataset such as the Breast Cancer Wisconsin dataset can be used for this practical.

Feature Category	Examples
Geometric measurements	Radius, Texture, Perimeter, Area
Shape characteristics	Smoothness, Compactness, Concavity, Concave points
Other characteristics	Symmetry, Fractal dimension
Target	Malignant or Benign

The dataset contains multiple numerical features, making it suitable for demonstrating dimensionality reduction using PCA.

The target variable represents the class of the observation, such as malignant or benign.

5. ALGORITHM

- Start.
- Load the high-dimensional dataset.
- Inspect the dataset and identify numerical features and the target variable.
- Check for missing, duplicate, and invalid values.
- Separate the input features from the target variable.
- Divide the dataset into training and testing sets.
- Standardize the numerical features.
- Apply PCA to the standardized training data.
- Analyze the explained variance ratio.
- Determine an appropriate number of principal components.
- Transform the training and testing data using the selected components.
- Record the number of original features and the number of retained components.
- Visualize the data using the selected reduced dimensions.
- Train a suitable classification model using the original features.
- Evaluate the classification model.
- Train the same classification model using the PCA-transformed features.
- Evaluate the classification performance after PCA.
- Compare the results before and after dimensionality reduction.
- Analyze whether PCA retains sufficient information for classification.
- Stop.

6. COMPARISON OF FEATURES BEFORE AND AFTER PCA

Parameter	Before PCA	After PCA
Number of features	Original high-dimensional feature set	Reduced number of principal components
Feature representation	Original variables	Principal components
Correlation	May contain correlated features	Components are orthogonal
Visualization	Difficult for high dimensions	Easier using 2D or 3D components
Computational complexity	Relatively higher	Reduced
Information	Contains original feature	Retains maximum variance

	information	through selected components

7. VISUALIZATION OF REDUCED DIMENSIONS

PCA allows high-dimensional data to be represented in two or three dimensions.

- A 2D visualization can be created using the first two principal components: Principal Component 1 (PC1) and Principal Component 2 (PC2).
- Different classes can be represented separately to examine whether the transformed features provide clear separation between classes.
- A 3D representation can also be considered using PC1, PC2, and PC3.

The visualization helps understand the distribution and separation of observations in the reduced feature space.

8. CLASSIFICATION PERFORMANCE EVALUATION

To analyze the impact of PCA, the same classification algorithm should be evaluated under two conditions:

- Classification using the original features.
- Classification using the PCA-transformed features.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

The performance before and after PCA should be compared to determine whether the reduced representation preserves sufficient information for classification.

9. OBSERVATION

- PCA reduces the number of dimensions while retaining the most significant variation in the dataset.
- The number of features after PCA is generally considerably smaller than the original feature set.
- Principal components are uncorrelated with each other.
- PCA makes high-dimensional data easier to visualize in two or three dimensions.
- Classification performance may remain similar when the selected components retain sufficient information.
- Excessive dimensionality reduction may result in information loss and a decrease in classification performance.

- PCA can reduce computational requirements by decreasing the number of features used by the classification model.
- The explained variance ratio helps determine an appropriate number of principal components.

10. APPLICATIONS OF PCA

- Image Processing: Reducing image feature dimensions.
- Pattern Recognition: Extracting important patterns from high-dimensional data.
- Medical Data Analysis: Reducing complex clinical and diagnostic features.
- Bioinformatics: Analyzing high-dimensional biological datasets.
- Computer Vision: Reducing image and feature representations.
- Data Visualization: Representing high-dimensional data in 2D or 3D.
- Machine Learning: Improving computational efficiency.
- Anomaly Detection: Identifying unusual observations in reduced feature spaces.
- Signal Processing: Extracting significant components from complex signals.

11. CONCLUSION

Principal Component Analysis is an effective dimensionality reduction technique for transforming high-dimensional data into a smaller set of informative principal components. By comparing the number of features before and after PCA, visualizing the reduced dimensions, and evaluating classification performance, the effectiveness of dimensionality reduction can be analyzed. PCA can significantly reduce computational complexity while preserving important information, provided that an appropriate number of principal components is selected.

PRACTICAL NO. 11

Advanced Machine Learning and Model Optimization (CO5)

1. AIM

To implement and compare Support Vector Machine (SVM), Random Forest, and AdaBoost classification algorithms, perform Cross Validation and Hyperparameter Tuning using Grid Search, and evaluate model performance before and after optimization.

2. OBJECTIVES

- To understand advanced supervised machine learning algorithms.
- To study the working principle of SVM, Random Forest, and AdaBoost.
- To implement different classification models on a suitable dataset.
- To evaluate the initial performance of the models.
- To perform Cross Validation for reliable model evaluation.
- To perform hyperparameter tuning using Grid Search.
- To compare model performance before and after optimization.
- To identify the most suitable optimized model based on evaluation metrics.

3. THEORY

3.1 Advanced Machine Learning

Advanced machine learning techniques provide effective approaches for solving complex classification and prediction problems. Different algorithms have different learning strategies, assumptions, strengths, and limitations.

In this practical, three classification algorithms are considered: Support Vector Machine (SVM), Random Forest, and AdaBoost. Their performance is evaluated before and after optimization.

3.2 Support Vector Machine (SVM)

Support Vector Machine is a supervised learning algorithm used for classification and regression. For classification, SVM attempts to find an optimal hyperplane that separates different classes.

The data points closest to the decision boundary are called support vectors.

The objective is to maximize the margin between the classes.

SVM can handle non-linear classification using kernel functions such as:

- Linear Kernel
- Polynomial Kernel

- Radial Basis Function (RBF)
- Sigmoid Kernel

Important SVM hyperparameters include:

- C: Controls the trade-off between a wider margin and classification errors.
- Gamma: Controls the influence of individual training samples for kernels such as RBF.
- Kernel: Specifies the mathematical function used to transform the data.

3.3 Random Forest

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to produce a more robust prediction.

Each decision tree is trained using a randomly selected sample of observations and features. The predictions from multiple trees are combined to obtain the final classification.

Important Random Forest hyperparameters include:

- Number of trees
- Maximum tree depth
- Minimum samples required for splitting
- Minimum samples required at a leaf
- Number of features considered for splitting

Random Forest generally reduces overfitting compared with an individual decision tree and can handle complex feature relationships.

3.4 AdaBoost

AdaBoost, or Adaptive Boosting, is an ensemble learning technique that combines multiple weak learners to form a stronger classifier.

Initially, equal importance is assigned to training observations. After each weak learner is trained, greater emphasis is given to incorrectly classified observations. Subsequent learners therefore focus more on difficult observations.

The final prediction is obtained by combining the predictions of the weak learners using weighted voting.

Important AdaBoost hyperparameters include:

- Number of estimators
- Learning rate
- Base estimator

4. DATASET

A suitable real-world classification dataset such as the Breast Cancer Wisconsin dataset can be used.

The dataset contains numerical characteristics computed from digitized images of breast mass samples.

Example features include:

- Radius
- Texture
- Perimeter
- Area
- Smoothness
- Compactness
- Concavity
- Concave points
- Symmetry
- Fractal dimension

The target variable represents two classes: Malignant and Benign.

The dataset is suitable for comparing multiple classification algorithms and studying the effect of model optimization.

5. CROSS VALIDATION

Cross Validation is a model evaluation technique used to estimate how well a machine learning model will perform on unseen data.

In K-Fold Cross Validation, the dataset is divided into K approximately equal-sized folds.

The process is:

- Divide the dataset into K folds.
- Use one fold as the validation set.
- Use the remaining K−1 folds for training.
- Train and evaluate the model.
- Repeat the process until every fold has been used as the validation set.
- Calculate the average performance across all folds.

Cross Validation provides a more reliable estimate of model performance than relying on a single train-test split.

6. HYPERPARAMETER TUNING USING GRID SEARCH

Hyperparameters are configuration values that are selected before the model training process.

Grid Search is a systematic hyperparameter optimization technique that evaluates different combinations of predefined hyperparameter values.

The general process is:

- Select the machine learning model.
- Define a range of hyperparameter values.
- Generate possible combinations of the selected values.
- Evaluate each combination using Cross Validation.
- Compare the validation scores.
- Select the combination providing the best performance.
- Train the optimized model using the selected hyperparameters.

Grid Search helps identify a better configuration of a machine learning model.

7. ALGORITHM

A. Initial Model Training

- Start.
- Load the real-world classification dataset.
- Inspect the dataset.
- Check for missing, duplicate, and invalid values.
- Separate input features and target variable.
- Perform necessary preprocessing.
- Divide the dataset into training and testing sets.
- Apply feature scaling where required.
- Train the SVM classifier using initial/default parameters.
- Train the Random Forest classifier using initial/default parameters.
- Train the AdaBoost classifier using initial/default parameters.
- Generate predictions for the test data.
- Calculate the selected evaluation metrics.
- Record the initial performance of all three models.

B. Cross Validation

- Select an appropriate K value for Cross Validation.

- Divide the training data into K folds.
- Train and validate each model using the K folds.
- Calculate the validation score for every fold.
- Calculate the average Cross Validation score.
- Compare the Cross Validation performance of SVM, Random Forest, and AdaBoost.

C. Hyperparameter Tuning using Grid Search

- Define suitable hyperparameter combinations for SVM.
- Perform Grid Search with Cross Validation.
- Identify the best SVM hyperparameters.
- Define suitable hyperparameter combinations for Random Forest.
- Perform Grid Search with Cross Validation.
- Identify the best Random Forest hyperparameters.
- Define suitable hyperparameter combinations for AdaBoost.
- Perform Grid Search with Cross Validation.
- Identify the best AdaBoost hyperparameters.
- Train the optimized models using their best hyperparameters.
- Generate predictions using the optimized models.
- Evaluate the optimized models.
- Compare performance before and after optimization.
- Identify the most suitable model.
- Stop.

8. MODEL EVALUATION METRICS

The following metrics can be used to evaluate and compare the models.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Accuracy represents the proportion of correctly classified observations.

Precision = TP / (TP + FP)

Precision measures the proportion of predicted positive observations that are actually positive.

Recall = TP / (TP + FN)

Recall measures the proportion of actual positive observations that are correctly

identified.

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

F1-Score provides a balance between Precision and Recall.

Confusion Matrix summarizes classification results using True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN).

9. COMPARISON BEFORE AND AFTER OPTIMIZATION

Model	Initial Model	Grid Search / Cross Validation	Optimized Model
SVM	Default parameters	Best parameter combination	Retrained optimized SVM
Random Forest	Default parameters	Best parameter combination	Retrained optimized Random Forest
AdaBoost	Default parameters	Best parameter combination	Retrained optimized AdaBoost

The comparison should consider Accuracy, Precision, Recall, F1-Score, Cross Validation score, and generalization performance.

10. OBSERVATION

- SVM, Random Forest, and AdaBoost provide different approaches to classification.
- Cross Validation provides a more reliable estimate of model performance.
- Grid Search systematically evaluates different hyperparameter combinations.
- Hyperparameter tuning can improve model performance compared with default configurations.
- The improvement obtained through optimization may vary for different algorithms.
- Random Forest can effectively capture complex relationships through multiple decision trees.
- SVM can provide strong classification performance when an appropriate kernel and parameter configuration are selected.
- AdaBoost improves classification by sequentially focusing on difficult observations.
- The optimized model should be selected based on overall evaluation performance rather than a single metric alone.

11. APPLICATIONS

- Healthcare: Disease diagnosis and medical classification.
- Financial Services: Credit risk and fraud detection.
- Cybersecurity: Intrusion and malicious activity detection.
- Marketing: Customer classification and segmentation.
- Manufacturing: Fault and quality detection.
- Education: Student performance prediction.
- Image Classification: Object and pattern recognition.
- Natural Language Processing: Text and document classification.
- Predictive Analytics: Business and operational decision-making.

12. CONCLUSION

SVM, Random Forest, and AdaBoost are powerful supervised machine learning algorithms that can be used for classification problems. Cross Validation provides a reliable method for evaluating model generalization, while Grid Search helps identify suitable hyperparameter configurations. By comparing the models before and after optimization, the effect of hyperparameter tuning on classification performance can be analyzed. The model with the best overall evaluation and generalization performance can then be selected for the given classification task.

MINI PROJECT – PRACTICAL NO. 12

AI/ML-Based Real-World Application (CO1–CO5)

1. AIM

To design and develop an AI/ML-based real-world application by defining a practical problem, collecting and preprocessing an appropriate dataset, developing a machine learning model, evaluating and optimizing its performance, and documenting the results.

2. OBJECTIVES

- To identify and define a real-world problem suitable for an AI/ML solution.
- To collect and understand an appropriate real-world dataset.
- To perform data cleaning and preprocessing.
- To select appropriate features and machine learning techniques.
- To develop and train suitable machine learning models.
- To evaluate model performance using appropriate evaluation metrics.
- To optimize the developed model using suitable techniques.
- To analyze the obtained results.
- To document the complete AI/ML-based solution.

3. PROJECT DOMAIN / APPLICATION AREAS

The mini project can be developed in any one of the following real-world application areas:

Sr. No.	Application	Objective
1	Disease Prediction System	Predict the possibility of a disease based on patient-related attributes.
2	Fraud Detection System	Identify potentially fraudulent transactions.
3	Customer Segmentation	Group customers according to their characteristics and behavior.
4	Recommendation System	Recommend relevant products, services, or content to users.
5	Student Performance Prediction	Predict student performance using academic and related attributes.
6	Crop Yield Prediction	Predict crop yield using agricultural and environmental factors.

The selected application should have a clearly defined problem, suitable dataset, measurable outcomes, and an appropriate machine learning approach.

4. PROJECT DEVELOPMENT PHASES

4.1 Problem Definition

The first phase is to clearly define the real-world problem that needs to be solved using AI/ML.

The problem definition should include:

- Background of the problem.
- Existing challenges.
- Need for an AI/ML solution.
- Input variables.
- Expected output.
- Type of machine learning problem.
- Scope of the proposed system.

Examples:

- Disease Prediction System: Predict whether a patient is likely to have a particular disease based on relevant medical attributes.
- Fraud Detection System: Identify whether a financial transaction is potentially fraudulent based on transaction-related characteristics.
- Customer Segmentation: Group customers into meaningful segments based on demographic and purchasing behavior.
- Recommendation System: Recommend relevant products, movies, books, or other items based on user preferences and historical interactions.
- Student Performance Prediction: Predict student performance using academic, attendance, assessment, and other relevant attributes.
- Crop Yield Prediction: Estimate crop yield based on factors such as crop type, rainfall, temperature, soil characteristics, and agricultural inputs.

4.2 Dataset Collection

An appropriate real-world dataset should be collected from a reliable source.

The dataset should be examined for:

- Number of records.
- Number of attributes.
- Data types.
- Target variable.

- Missing values.
- Duplicate records.
- Outliers.
- Categorical variables.
- Numerical variables.
- Class distribution, where applicable.

Possible dataset sources include publicly available machine learning repositories, government datasets, research datasets, and institutional datasets.

The selected dataset should be relevant to the defined problem.

4.3 Data Preprocessing

Data preprocessing prepares the collected dataset for machine learning.

Data Cleaning

- Identify missing values.
- Handle duplicate records.
- Correct inconsistent values.
- Remove irrelevant attributes.
- Handle invalid observations.

Missing Value Handling

- Mean imputation.
- Median imputation.
- Mode imputation.
- Forward or backward filling where appropriate.
- Removal of records when justified.

Encoding

- Label Encoding.
- One-Hot Encoding.

Feature Scaling

- Standardization.
- Min-Max Normalization.

Feature Selection

Relevant features should be selected while removing redundant or irrelevant attributes.

Train-Test Split

The dataset should be divided into training and testing sets to evaluate the generalization ability of the developed model.

5. MODEL DEVELOPMENT

After preprocessing, appropriate machine learning algorithms should be selected according to the problem type.

Classification Problems

- Logistic Regression.
- K-Nearest Neighbors.
- Decision Tree.
- Random Forest.
- Support Vector Machine.
- Naïve Bayes.
- AdaBoost.

Regression Problems

- Linear Regression.
- Polynomial Regression.
- Decision Tree Regression.
- Random Forest Regression.
- Support Vector Regression.

Clustering Problems

- K-Means Clustering.
- Hierarchical Clustering.

Recommendation Systems

- Content-Based Filtering.
- Collaborative Filtering.
- Hybrid Recommendation.

The selection of the algorithm should be justified based on the characteristics of the selected problem and dataset.

6. PERFORMANCE EVALUATION

The developed model should be evaluated using appropriate performance metrics.

For Classification

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- ROC-AUC, where appropriate

For Regression

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

For Clustering

- Silhouette Score
- Within-Cluster Sum of Squares (WCSS)
- Cluster interpretation

For Recommendation Systems

- Precision
- Recall
- F1-Score
- Recommendation relevance
- User-item interaction performance

The evaluation metric should be selected according to the nature of the project.

7. MODEL OPTIMIZATION

Model optimization is performed to improve the performance and generalization capability of the developed system.

- Hyperparameter tuning.
- Grid Search.
- Random Search.
- Cross Validation.
- Feature selection.

- Feature engineering.
- Regularization.
- Ensemble techniques.
- Appropriate threshold selection.

The performance of the model before and after optimization should be compared.

The optimized model should not only provide improved training performance but should also demonstrate good performance on unseen data.

8. RESULT ANALYSIS

The results obtained from the developed AI/ML system should be analyzed systematically.

The analysis should include:

- Dataset characteristics.
- Important features identified during analysis.
- Model performance.
- Comparison of different algorithms, where applicable.
- Performance before optimization.
- Performance after optimization.
- Effect of preprocessing.
- Effect of feature selection or engineering.
- Strengths and limitations of the developed system.
- Practical interpretation of the results.

A suitable comparison table can be prepared:

Model	Evaluation Metric 1	Evaluation Metric 2	Evaluation Metric 3	Overall Observation
—	—	—	—	—
—	—	—	—	—
—	—	—	—	—
—	—	—	—	—

The actual values should be filled based on the results obtained from the selected dataset and models.

9. DOCUMENTATION

The complete mini project should be documented systematically.

9.1 Introduction

- Background of the selected problem.
- Motivation.
- Need for the proposed system.

9.2 Problem Statement

Clearly defined real-world problem.

9.3 Objectives

Specific objectives of the project.

9.4 Dataset Description

- Dataset source.
- Number of records.
- Attributes.
- Target variable.
- Data characteristics.

9.5 Data Preprocessing

- Data cleaning.
- Missing value handling.
- Encoding.
- Feature scaling.
- Feature selection.
- Dataset splitting.

9.6 Model Development

- Selected algorithms.
- Reason for algorithm selection.
- Model development process.

9.7 Performance Evaluation

- Evaluation metrics.
- Model comparison.
- Validation results.

9.8 Model Optimization

- Hyperparameter tuning.

- Cross Validation.
- Optimized parameters.
- Comparison before and after optimization.

9.9 Result Analysis

- Results obtained.
- Interpretation.
- Comparison of models.
- Key findings.

9.10 Conclusion

- Summary of the developed system.
- Effectiveness of the proposed AI/ML approach.
- Possible future improvements.

10. OBSERVATION

- A well-defined problem is essential for developing an effective AI/ML application.
- Dataset quality directly affects the performance of the machine learning model.
- Data preprocessing improves the quality and consistency of the input data.
- Different machine learning algorithms may produce different performance for the same dataset.
- Appropriate evaluation metrics should be selected according to the application.
- Hyperparameter optimization can improve model performance and generalization.
- Cross Validation provides a more reliable assessment of model performance.
- Result analysis helps determine whether the developed system is suitable for the intended real-world application.

11. APPLICATIONS

- Healthcare and disease diagnosis.
- Banking and financial fraud detection.
- Customer analytics and marketing.
- E-commerce recommendation systems.
- Educational analytics.

- Agriculture and crop prediction.
- Business intelligence.
- Cybersecurity.
- Industrial automation.
- Predictive analytics.

12. CONCLUSION

The AI/ML-based mini project provides an opportunity to apply concepts learned throughout the course to a real-world problem. The project covers the complete machine learning lifecycle, starting from problem definition and dataset collection, followed by data preprocessing, model development, performance evaluation, model optimization, result analysis, and documentation. By selecting an appropriate application such as disease prediction, fraud detection, customer segmentation, recommendation, student performance prediction, or crop yield prediction, the effectiveness of AI/ML techniques can be demonstrated in a practical real-world.